Passive Waiting and Mutexes

Introduction to Critical Sections and Waiting

- Entry: Thread blocks until the critical section is free
- Critical Section: Actual usage of the shared resource
- Exit: Grants access to other waiting or future threads
- Active Waiting: Continuous testing of variables for entry
- Passive Waiting: Thread relinquishes CPU and becomes non-schedulable

■ *A flow diagram showing Entry -> Critical Section -> Exit*

Note: Emphasize the CPU cost difference between the two waiting strategies. Active waiting (spinning) consumes CPU cycles, while passive waiting is more efficient for the processor but involves the overhead of relinquishing and later regaining the CPU. Mention that mutexes and semaphores are primary examples of passive waiting solutions.

Mechanics of Passive Waiting

- Threads block and relinquish CPU when CS is unavailable
- Blocked threads become non-schedulable and enter sleeping state
- Exiting the CS restores the condition for entry
- The exiting thread wakes up one waiting or sleeping thread

■ *Illustration of a thread moving from 'Ready Queue' to 'Sleeping State'*

Note: Contrast this with active waiting. Emphasize that passive waiting is more CPU-efficient because the thread is removed from the scheduler's ready queue. When the thread in the critical section exits, it is responsible for triggering the wake-up mechanism for a waiting thread.

Mutexes: The Passive Waiting Solution

- Mutexes are passive waiting solutions for mutual exclusion
- Includes pthread locks and Java's synchronized keyword
- Costly due to relinquishing and regaining the CPU
- Only one thread can own the mutex at a time

■ *Icon of a lock and key representing a Mutex*

Note: Emphasize the trade-off between ease of use and performance. Explain that 'passive waiting' means the thread isn't spinning (busy-waiting) but is instead put to sleep by the OS, which is why it is described as 'costly' in terms of CPU overhead.

The Danger of Deadlocks

- Deadlock: two or more threads circularly blocking each other
- Occurs when threads wait for resources held by others
- Results in a total halt as no thread can proceed
- Detected by identifying cycles in a 'waiting for resources' graph

■ *A circular diagram showing Thread A waiting for B and B waiting for A*

Note: Explain that deadlocks are 'fatal' because the threads aren't just slow—they are completely stopped. Use the array example to illustrate the 'circular' nature of the dependency. Mention that while the OS can detect this via resource graphs, a common prevention strategy is to impose a strict ordering on how resources are requested (e.g.,

always lock A before B).

Deadlock Detection and Prevention

- Deadlocks occur when activities hold resources while waiting
- Waiting for resources graphs are used for deadlock detection
- A cycle in the resource graph indicates a deadlock
- Resource ordering prevents deadlocks by eliminating circular waits

■ *A directed graph showing a cycle vs. a linear resource path*

Note: Start by explaining the 'deadly embrace' scenario using the array copy example. Emphasize that the non-deterministic nature of CPU scheduling is what makes these bugs hard to find. When discussing the graph, explain that a cycle is